Bytemap: A visual memory & disk inspector

Semester Project Proposal

Group Members:

241815- Muhammad Usaidullah Rehan

241887- Insha Khan

241866- Aniqa Irshad

Submitted to: Romana Maroof

Submission date: 2 May, 2026

SEMESTER PROJECT PROPOSAL

ByteMap: A Visual Memory & Disk Inspector

1. Chosen Domain

ByteMap is a system-level utility that operates directly at the hardware layer, accessing raw RAM contents and raw disk sectors without any operating system abstraction. It communicates with hardware exclusively through BIOS interrupts, making it a genuine low-level assembly language application that demonstrates the core principles taught in the COAL course.

2. Project Objective

The objective of ByteMap is to design and implement a fully functional visual memory and disk inspection tool in 8086 Assembly Language. The program will allow the user to read, view, and edit raw bytes stored in RAM and on disk, displayed through a color-coded graphical interface running in EMU8086. ByteMap will demonstrate key COAL concepts including memory segmentation, BIOS interrupt handling, modular procedure-based programming, stack frame conventions, and VGA graphics output.

3. Specific Functionality

- **RAM Mode:** Reads live memory using DS and ES segment registers and displays every byte as a color-coded hex value on screen. The user can move a cursor to any byte and overwrite it instantly using MOV [BX], AL.
- **Disk Mode:** Uses INT 13h (BIOS disk interrupt, Service 02h) to read a raw 512-byte sector from the virtual floppy disk into a buffer. The user can edit any byte on screen and write changes back to the sector (Service 03h), or wipe the sector by filling it with zeros.
- **VGA Graphics Engine:** Uses INT 10h (Mode 13h) to switch into graphics mode and draw a color-coded hex grid. Each byte type is null, printable ASCII, executable instruction, or other data is assigned a unique color for immediate visual identification.
- **Keyboard Navigation:** Arrow keys move a visible cursor across the hex grid using INT 16h. The user selects bytes, edits values, and switches between RAM and Disk modes through an on-screen menu.
- **Hex Conversion:** Binary-to-hex conversion throughout the program is handled using bitwise SHR, SHL, and AND instructions. No library functions used.

4. Expected Challenges

- Correctly computing and managing segment:offset addresses in 16-bit real mode for accurate memory reading and writing.
- Implementing stable VGA graphics output with cursor movement in Mode 13h without screen flicker or corruption.
- Passing disk sector parameters (drive number, cylinder, sector, head) to procedures via proper stack frame conventions as required by the course.
- Reliable INT 13h disk read/write operations within the EMU8086 virtual environment and handling error codes gracefully.

5. Potential Benefits

- Provides direct hands-on experience with 8086 memory segmentation, addressing modes, and hardware-level data access the core of the COAL syllabus.
- Reinforces all major course objectives: stack frames, modular procedures, macros, BIOS interrupts, graphics, and low-level debugging.
- Produces a genuinely useful educational tool that visually demonstrates how data is stored at the byte level in both volatile and non-volatile memory.
- Develops confidence and practical skill in low-level systems programming applicable to operating systems, embedded systems, and device driver development.